SOFTWARE ENGINEERING / RESEARCH PROJECT

CHECKLIST SUBMISSION

PP1 + PP2 | Evidence and Deployment Report

R26-SE-013 • ADRS: A Centralized System for Real-Time Farm Animal Disease Reporting

DOCUMENT CONTROL	DETAIL
Project ID	R26-SE-013
Project / system	Animal Disease Reporting System (ADRS)
Submission areas	Checklist 1: Git repository (PP1) | Checklist 2: UI/UX demo video (PP2) | Checklist 3: deployment report (PP2)
Supervisor	Dilani Lunugalage
Evidence cut-off	13 October 2026

Research group

STUDENT	STUDENT ID	RESEARCH COMPONENT
A.L.M. Athulathmudali	IT21129544	Computer Vision Diagnostic & Prognostic Assessment
G.A. Sandaru	IT22258908	Real-Time Geospatial Clustering & Trajectory Mapping
S.S. Kumarasinghe	IT22221414	Climate-Informed Seasonal Livestock Disease Forecasting
W.M.P.J. Wijenayake	IT22194558	AI-Driven Wellness and Management Standardization

1 Submission overview

SUBMISSION	REQUIRED ARTIFACT	EVIDENCE POSITION
Checklist 1 · PP1	Git repository	Repository, branches, README, component code and commit evidence confirmed at review date. Historical state at the PP1 presentation must be independently shown if required.
Checklist 2 · PP2	UI/UX demo video	Interface screens and planned video coverage documented
Checklist 3 · PP2	Deployment report	Report drafted from the code and documented local setup.

1.1 Key submission links

GitHub repository: https://github.com/lasitha-dev/R26-SE-013

Default branch: https://github.com/lasitha-dev/R26-SE-013/tree/main

Latest examined integration merge: https://github.com/lasitha-dev/R26-SE-013/commit/8a778207d3f892534a7efdd898c5a9ff04803f31

UI/UX video link **https://drive.google.com/file/d/1Jq0lAal_YEdIpo1OkgQbvRrjojQpIEFj/view?usp=sharing**

2 Checklist 1 — Git repository

Submission milestone: by the PP1 presentation. The table below establishes the available GitHub evidence as of 13 October 2026. The April log records repository initiation.

2.1 Git repository evidence checklist

CHECK	EVIDENCE / EXPECTED DELIVERABLE	STATUS
01	Repository accessible at github.com/lasitha-dev/R26-SE-013	VERIFIED IN REPO
02	README identifies project, four components, tech stack and local setup	VERIFIED IN REPO
03	Separate backend/ and frontend/ directories and component structure	VERIFIED IN REPO
04	Four research-domain backend directories: smart_diagnostics, geospatial_tracking, risk_forecasting, health_anomaly	VERIFIED IN REPO
05	Corresponding frontend feature folders and routed application views	VERIFIED IN REPO
06	Component-specific branches and an integration branch in Git history	VERIFIED IN REPO
07	Visible commit history, including geospatial integration and live outbreak/forecast changes	VERIFIED IN REPO
08	Backend/Frontend test files included in source tree	VERIFIED IN REPO

2.2 Repository layout and ownership

RESEARCH COMPONENT	REPOSITORY EVIDENCE	STUDENT
Smart diagnosis	backend/components/smart_diagnostics/ frontend/src/features/SmartDiagnostics/	A.L.M. Athulathmudali
Geospatial intelligence	backend/components/geospatial_tracking/ frontend/src/features/GeospatialMap/	G.A. Sandaru
Seasonal forecasting	backend/components/risk_forecasting/ frontend/src/features/RiskForecasting/	S.S. Kumarasinghe
Wellness & management	backend/components/health_anomaly/ frontend/src/features/HealthAnomaly/	W.M.P.J. Wijenayake

2.3 Git reference and review notes

Repository URL https://github.com/lasitha-dev/R26-SE-013

Default branch main; current reviewed head: 8a778207d3f892534a7efdd898c5a9ff04803f31

Example component branches component/ai-smart-diagnosis; component/geo-spatial-visualization; component/disease-forecasting; component/ai-driven-wellness-management

Integration branch integrate/updated-latest

2.4 Direct GitHub source links

Verified paths in the public main branch and currently listed component branches; links below lead to inspectable source evidence, not deployed runtime proof.

VERIFIED REPOSITORY EVIDENCE	DIRECT LINK
Repository / main	Open repository
Project README	View README
Component branches	View branches
Smart diagnosis – backend	Open module
Geospatial intelligence – backend	Open module
Seasonal forecasting – backend	Open module
Animal wellness – backend	Open module
Frontend views / routes	View App.jsx
Farmer dashboard routes	View routes
Backend route composition	View main.py
Frontend dependency and test scripts	View package.json
Main branch integration commit	8a778207d3f8

3 Checklist 2 — UI/UX demo video

Submission milestone: by the PP2 presentation. The video should demonstrate the actual working interface, navigation, representative input/output and observed limitations. This section provides a recording plan and evidence register. A narrated screenshot-based MP4 was prepared separately; it is not evidence of live user interactions.

3.1 Video submission metadata

FIELD	TO BE COMPLETED
Video title	R26-SE-013 | ADRS — PP2 UI/UX Demonstration
Duration	07:48
Demonstrated build	8a778207d3f892534a7efdd898c5a9ff04803f31
Url	https://drive.google.com/file/d/1Jq0IAal_YEdIpo1OkgQbvRrjojQpIEFj/view?usp=sharing

3.2 Screen traceability to checked-in code

The route declarations in frontend/src/App.jsx and frontend/src/features/HealthAnomaly/HealthAnomalyRoutes.jsx include farmer, veterinary, geospatial and forecasting navigation.

- Farmer views: /health/dashboard, /health/herd-registry, /health/7-day-triage-scan, /health/bcs-analyzer, /health/diagnostics.
- Veterinary views: /vet/login and authenticated /vet routes; access requires relevant role and test credentials.
- Forecasting demo: /risk-forecasting-demo is explicitly feature-flagged and disabled unless VITE_FORECASTING_DEMO_ENABLED=true.
- Capture usable evidence: show the screen, the test input, the resulting response and a short explanation of clinical/research limitations.

4 Checklist 3 — Deployment report

Submission milestone: by the PP2 presentation. This report documents the verifiable implementation architecture, how the supplied code is intended to run, and the evidence still needed to certify an actual deployment.

4.1 Deployment architecture

LAYER	TECHNOLOGY / RESPONSIBILITY	SOURCE
Web client	React 18, Vite 5, Tailwind CSS; route-based farmer/vet/DAPH views and four research features	frontend/package.json; frontend/src/App.jsx
Backend API	Python FastAPI + Uvicorn; integrates diagnostics, wellness, forecasting and geospatial services	backend/main.py; backend/requirements.txt
AI inference	YOLO/ViT and segmentation wrappers, plus ML forecast and wellness processing modules	backend/components/
Persistence	MongoDB connection and repositories for operational records and analysis	backend/core/; README.md
Local networking	Frontend dev proxy /api -> http://localhost:8000; backend commonly served on port 8000	frontend/vite.config.mjs; README.md
Test tooling	Vitest + Testing Library for UI, Python test modules for API/component paths	frontend/package.json; backend test files

4.2 Local operating environment (documented)

PARAMETER	DOCUMENTED BASELINE
Backend runtime	Python 3.10+ and a dedicated virtual environment
Frontend runtime	Node.js 18+; npm dependencies and Vite
Database	MongoDB Atlas or locally configured MongoDB, with environment-based URI
Backend address	http://localhost:8000; API docs at http://localhost:8000/docs
Frontend dev address	http://localhost:5173
Production hosting / domain	No verified public hosting URL; local development documented only

4.3 Environment settings

The README mentions MONGO_URI, while backend/.env.example shows MONGODB_URL and MONGODB_DB_NAME. The deployment operator must confirm the effective variable name from backend/core/database.py and use one consistent configuration for the actual running build.

4.4 Reproducible local setup procedure

The following steps are based on the repository README, package scripts and backend entry point. They are preparation steps, not a claim that this report author executed the application.

Backend service

```
cd backend
python -m venv venv
venv\Scripts\activate  # Windows; on macOS/Linux: source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --host 127.0.0.1 --port 8000
```

Frontend service

```
cd frontend
npm ci
Copy frontend/.env.example to frontend/.env and set VITE_API_URL
npm run dev  # development mode
npm run build  # prepare distributable assets
```

4.5 Smoke test and sign-off evidence

CHECK	EXPECTED RESULT
API root	GET http://localhost:8000/ returns JSON; database_connected reflects true/false
API docs	/docs loads OpenAPI documentation and registered endpoints
Frontend route	Browser serves the application and login/dashboard is visible
Core API integration	Representative image/wellness/map/forecast requests return expected schema
Frontend build	npm run build completes with exit code 0
Frontend tests	npm test executes and results are recorded
Backend tests	Selected pytest suites run with results recorded
Restart/recovery	Services restart and reconnect after planned restart

4.6 Deployment platform details — completion fields

REQUIRED EVIDENCE	VALUE / ATTACHMENT
Environment	Local development documented; cloud/staging not confirmed
Hosting provider and region	Not specified or verified in the repository
Frontend HTTPS URL	Not available in GitHub; local dev: http://localhost:5173 (not public)
Backend HTTPS URL / API base	Not available in GitHub; local API: http://localhost:8000 (not public)
Repository branch, deployed commit SHA	main commit 8a778207d3f8 (repository head only; deployment SHA unverified)
Deployment date/time and operator	No deployment date/operator record identified in repository
Deployment log, screenshots and health proof	No external deployment logs/health screenshots identified; attach separately

4.7 Production-readiness observations

The following items are source-review observations to resolve or explicitly document before claiming secure production deployment. They are not a security audit or a runtime penetration test.

PRIORITY	OBSERVATION	REQUIRED ACTION
High	backend/main.py includes permissive development CORS origins, including "*".	Replace with approved HTTPS frontend origins for production and verify cross-origin behaviour.
High	A frontend/.env.local file is present in tracked repository contents.	Audit for secrets, remove sensitive tracked values and rotate credentials if any were committed.
Medium	Repository tree includes frontend/node_modules/ and frontend/dist/.	Confirm build-artifact policy; prefer dependency lockfile and CI-built artifacts rather than committing generated dependencies.
Medium	The API root reports database errors; the global exception handler includes error strings.	Avoid exposing internal error details to untrusted clients in production.
Medium	The model pipeline depends on large ML packages/models and external services.	Verify model-file paths, dependency compatibility, CPU/GPU resources and health probes on the chosen host.
Medium	No public deployment endpoint or production operations evidence was supplied.	Attach hosting details, HTTPS/health check, logs and a dated PP2 screenshot when available.

4.8 Operational safeguards to document

- Secrets and data: use environment/secret management, restricted MongoDB network access and least-privilege service accounts.
- Deployment: document build and restart commands, uptime monitoring, log access and an explicit rollback route.
- Data integrity: show evidence that geospatial and forecasting analyses use their intended historical/live data sources and mark synthetic demonstration records.
- Responsible use: model predictions serve screening and surveillance support; clinical confirmation remains with authorized veterinary officers.

4.9 Formal deployment conclusion

ADRS has a repository-based React/FastAPI application architecture with four implemented research-domain code areas and documented local development steps. The current documentation is sufficient to describe the intended deployment process, but not to certify an external production release. Checklist 3 should be marked complete only after the team supplies the host, actual deployed commit, launch/health logs, representative test outputs and approval evidence.

5 Traceability and evidence register

This table links checklist statements to the supplied logs, planner and GitHub sources. Page ranges refer to the uploaded documents, not to the page numbers of this report.

SOURCE	DATE / LOCATION	TRACEABLE CONTENT
[L1] Computer vision log	April–August 2026; pp. 1–15	Dataset/YOLO/ViT work; endpoints, frontend, tests and integration entries. Dated 28 Aug end-to-end demo verification is recorded.
[L2] Geospatial log	April–August 2026; pp. 1–13	ST-DBSCAN, trajectories, environmental risk, geospatial APIs, maps and tested integration tasks.
[L3] Forecasting log	April–August 2026; pp. 1–14	Data preparation, ML forecast and severity models, forecast APIs, integration and validation entries.
[L4] Wellness log	April–August 2026; pp. 1–13	BCS images, 7-day inputs, THI, React screens, Grad-CAM and data-driven dashboard.
[P1] Group planner	R26-SE-013 workbook; export 13 Oct 2026	49 populated tasks: 36 Completed and 13 Not started; statuses may be outdated relative to source code.
[G1] GitHub repository	main branch inspected 13 Oct 2026	Current files, README, branches, source modules and commit evidence.

5.1 Dated technical milestones from research logs

DATE / PERIOD	ILLUSTRATIVE WORK RECORDED
Apr–May 2026	Repository/project structure, initial models, UI prototypes and proof-of-concept demonstrations.
Jun–Jul 2026	API schemas and routers, image processing, geospatial clustering, wellness inference and forecasting integration.
Aug 2026	Component-level testing, UI and model-output improvements, integration and evaluation preparation.
13 Oct 2026	Current repository and planner reviewed for preparation of this submission portfolio.

Main repository · Open repository on GitHub

Readme · Open README.md

Backend app entry point · Open backend/main.py

Frontend route definitions · Open frontend/src/App.jsx

Latest inspected integration commit · Open commit 8a778207d3f8

Latest inspected integration commit · Open source

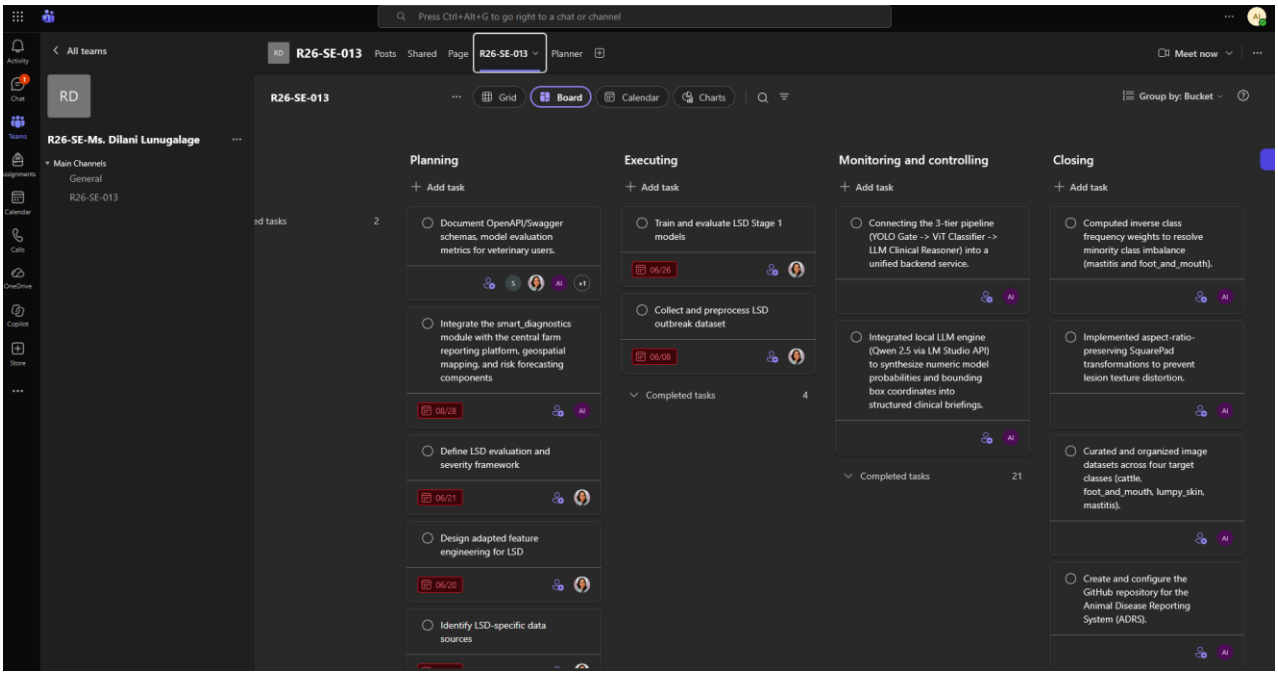


Figure 1 Teams Planner

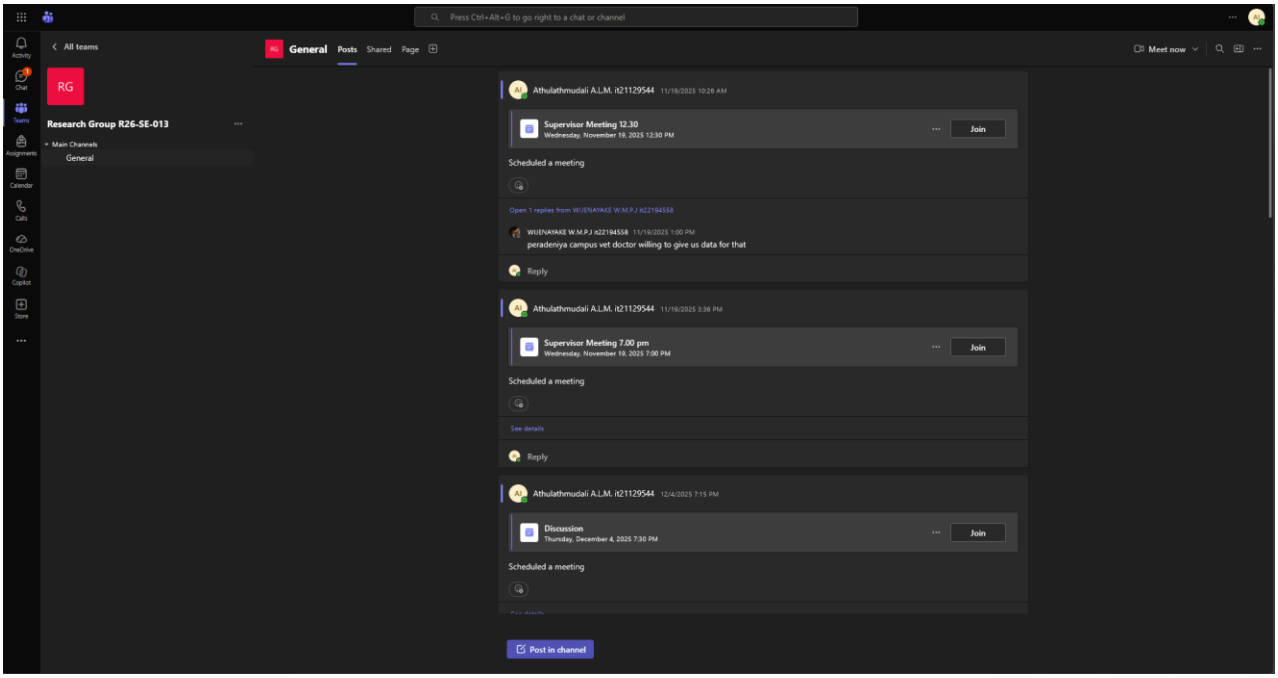


Figure 2 Discussions with the Supervisor